

“HOSTEL & MESS MANAGEMENT SYSTEM”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

KARNATI LAXMI LAHARI 2520090206

SIVARAPU RISHIL 2520090174

Under the Esteemed Guidance of

Dr. Khaja Shareef

Associate Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Hostel & Mess Management System”** is a record of bonafide work of **KARNATI LAXMILAHARI – 2520090206, SIVARAPU RISHIL - 2520090174** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 1, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

KARNATI LAXMI LAHARI – 2520090206

SIVARAPU RISHIL – 2520090174

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“Hostel & Mess Management System”** is a bonafide work done and submitted by **KARNATI LAXMILAHARI – 2520090206, SIVARAPU RISHIL – 2520090174** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 12, Team 1, at KLH University during **2026–2027, Trimester 4.**

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr** , and my guide **Khaja Shareef**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the *Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

KARNATI LAXMI LAHARI – 2520090206

SIVARAPU RISHIL – 2520090174

Abstract

The **Hostel & Mess Management System** is a web-based application designed to digitalize and simplify hostel and mess-related activities in educational institutions. Traditional hostel management often depends on manual registers, spreadsheets, and separate communication channels for managing student information, room allocation, complaints, leave requests, mess attendance, payments, and administrative activities. Such approaches can result in data duplication, delays, allocation errors, and difficulty in maintaining accurate records.

The proposed system provides a centralized platform through which **students, wardens, and administrators** can perform their respective hostel and mess operations. Students can securely log in, view their profile and room information, check mess menus, record meal attendance, submit complaints, request leave, view notifications, and check payment information. Wardens can monitor students, rooms, room allocations, complaints, and leave requests. Administrators can manage students, wardens, hostels, rooms, mess menus, payments, and system reports.

The application uses a **React.js frontend with Vite**, a **Node.js and Express.js backend**, and a **MySQL relational database**. Axios is used for communication between the frontend and backend through REST APIs. Authentication is implemented using **JSON Web Tokens (JWT)**, while passwords are securely hashed using **bcryptjs**. Role-based authorization ensures that students, wardens, and administrators can access only the functionality permitted for their respective roles.

The relational database maintains structured relationships between users, students, wardens, hostels, rooms, room allocations, mess menus, mess attendance, complaints, leave requests, payments, and notifications. Primary keys, foreign keys, unique constraints, transactions, and validation logic are used to maintain data consistency.

The system was developed incrementally and tested through backend API testing, database verification, frontend testing, authentication testing, and role-based access testing. The completed application provides a centralized and organized platform for hostel and mess management while reducing dependence on manual processes.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	9
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	10
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	12
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	13
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	15
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	17
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	19
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	21
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	22

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	23
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	24
13	References	– Documentation, tools, and resources used	25
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	26

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Hostel management in educational institutions involves several activities including student registration, room allocation, hostel administration, mess management, complaints, leave requests, payments, and maintaining student records.

In many institutions, these activities may be handled using manual registers, spreadsheets, paper forms, or separate communication systems. Managing these records manually becomes difficult as the number of students and hostel facilities increases. Room availability may not be immediately visible, complaints can be difficult to track, and leave approvals may require repeated communication between students and wardens.

The **Hostel & Mess Management System** was developed to provide a centralized digital platform for managing these activities.

The system maintains hostel information, rooms, student records, room allocations, mess menus, attendance, complaints, leave requests, payments, and notifications in a relational MySQL database.

Instead of maintaining separate records, the system provides different dashboards for **Students, Wardens, and Administrators**, allowing each role to perform its authorized operations.

1.2 Problem Statement

Traditional hostel management methods can create several problems:

- Manual maintenance of student and room records.
- Difficulty identifying available rooms.
- Errors during room allocation and transfers.
- Difficulty tracking complaints and their status.
- Manual processing of leave requests.
- Separate or inconsistent mess attendance records.
- Difficulty monitoring hostel and mess payments.
- Lack of centralized notifications.
- Unauthorized access to administrative information.
- Difficulty generating consolidated reports.

Therefore, there is a need for a centralized system that can securely store hostel information and automate common hostel and mess management operations.

1.3 Scope of the Project

The system covers the major hostel and mess management activities required for a college environment.

The scope includes:

- Student registration and authentication.
- Student profile management.
- Hostel and room management.

- Room allocation and deallocation.
- Room transfer functionality.
- Mess menu management.
- Mess attendance management.
- Complaint submission and status tracking.
- Leave request submission and approval.
- Hostel and mess payment management.
- Notifications.
- Warden dashboard.
- Administrator dashboard.
- Reports and system statistics.
- Role-based access control.

The current implementation is designed primarily for local development and academic demonstration. Cloud deployment, mobile applications, biometric attendance, and automated payment gateways are considered future enhancements.

1.4 Objectives

The main objectives of the project are:

- To develop a centralized hostel management platform.
- To reduce dependency on manual hostel records.
- To provide accurate room availability information.
- To simplify room allocation and transfer.
- To digitalize complaint management.
- To automate the leave request workflow.
- To manage mess menus and attendance digitally.
- To provide payment information in a centralized system.
- To implement secure authentication using JWT.
- To implement role-based authorization for students, wardens, and administrators.
- To provide administrators with centralized reports.
- To improve data consistency using a relational database.

1.5 Importance of the Application

The application provides a centralized source of information for hostel operations.

Students can access their hostel information without depending on manual registers. Wardens can monitor students, rooms, complaints, and leave requests through a single dashboard. Administrators can manage hostel infrastructure, users, payments, mess information, and reports.

The system also improves data consistency by storing related information in a structured relational database.

1.6 Target Users / Audience

Student

Students can:

- View their profile.
- View room details.
- View mess menus.
- View attendance.
- Submit complaints.
- Submit leave requests.
- View payment information.
- View notifications.

Warden

Wardens can:

- View students.
- Monitor rooms.
- Allocate and transfer rooms.
- View complaints.
- Update complaint status.
- Review leave requests.
- Approve or reject leave requests.

Administrator

Administrators can:

- Manage students.
- Manage wardens.
- Manage hostels.
- Manage rooms.
- Manage mess menus.
- Manage payments.
- View reports.
- Monitor overall hostel operations.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application has relatively modest hardware requirements.

Recommended development environment:

- Dual-core processor or higher.
- Minimum 4 GB RAM.
- At least 2 GB available storage for project files and dependencies.
- Stable internet connection for package installation and GitHub operations.
- Keyboard and mouse.
- Display monitor.

The complete application can be developed and demonstrated on a single computer

2.2 Software Requirements

- macOS / Windows / Linux
- Node.js
- npm
- MySQL 8.x
- Visual Studio Code
- Modern web browser
- Git
- GitHub
- MySQL command-line client or MySQL Workbench
- Thunder Client / Postman for API testing

2.3 Development Tools and Frameworks

The project uses:

- **React.js + Vite** for the frontend.
- **Node.js + Express.js** for the backend.
- **MySQL** for relational data storage.
- **mysql2** for Node.js–MySQL communication.
- **Axios** for API communication.
- **React Router** for frontend navigation.
- **JWT** for authentication.
- **bcryptjs** for password hashing.
- **Git + GitHub** for version control.
- **Thunder Client** for API testing.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is implemented using **React.js** and **Vite**.

React provides component-based development, while React Router handles navigation between different dashboards and pages.

Axios is used to communicate with the backend REST APIs.

The frontend contains separate interfaces for:

- Student
- Warden
- Admin

Reusable components such as sidebars, navigation components, protected routes, loading components, and notifications are used throughout the application.

3.2 Backend

The backend is developed using **Node.js** and **Express.js**.

The backend follows a modular structure containing:

- Routes
- Controllers
- Middleware
- Database configuration
- Utility functions

The backend exposes REST APIs for authentication, students, hostels, rooms, room allocations, mess, complaints, leaves, payments, notifications, wardens, and administrators.

3.3 Database

The system uses **MySQL** as its relational database.

The major tables are:

1. users
2. students
3. wardens
4. hostels
5. rooms
6. room_allocations
7. mess_menu
8. mess_attendance
9. complaints

- 10. leave_requests
- 11. payments
- 12. notifications

Primary keys and foreign keys are used to establish relationships between entities.

3.4 Version Control

Git is used for source-code version control.

The project is maintained in Git and uploaded to GitHub in the existing **Database** repository under the Hostel & Mess Management System project.

Version control allows changes to be tracked and provides a centralized location for the project source code.

3.5 Authentication and Security Libraries

Authentication is implemented using **JSON Web Tokens (JWT)**.

Passwords are hashed using **bcryptjs** before being stored in the database.

Protected backend routes verify the JWT before allowing access.

Role-based middleware restricts access according to the user's role:

```
Student → Student APIs
Warden  → Warden APIs
Admin   → Admin APIs
```

The frontend also uses `ProtectedRoute` to prevent unauthorized dashboard access.

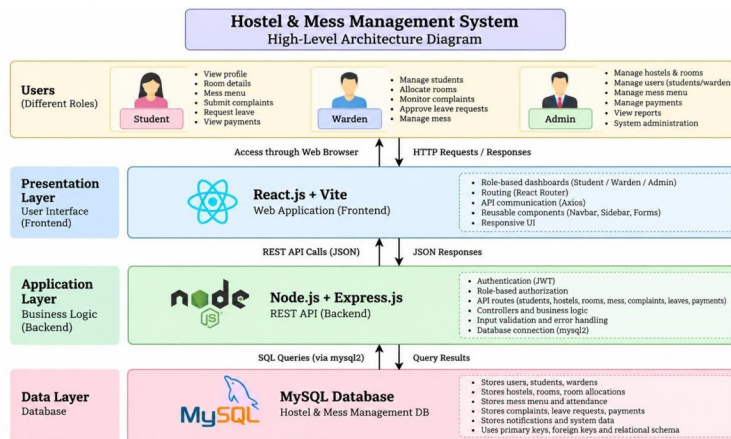
CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

Presentation Layer: React.js + Vite frontend for Students, Wardens, and Admins.

Application Layer: Node.js + Express.js handles REST APIs, authentication, and business logic.

Data Layer: MySQL stores hostel, student, room, mess, complaint, leave, payment, and notification data.



4.2 Description of Each Layer

Presentation Layer

The presentation layer is implemented using React and Vite. It provides separate interfaces for students, wardens, and administrators. The frontend communicates with the backend through HTTP REST APIs and does not directly access the MySQL database.

Application Layer

The application layer consists of Node.js and Express.js. Requests pass through authentication and role middleware before reaching the relevant controller. Controllers perform validation, execute business logic, and communicate with MySQL.

Data Layer

The data layer consists of MySQL. The database stores users, students, wardens, hostels, rooms, allocations, mess information, complaints, leave requests, payments, and notifications.

4.3 Deployment Architecture

The system is deployed locally for development and demonstration. The React frontend runs on port 5173, the Node.js/Express backend runs on port 5002, and MySQL runs on port 3306. The frontend communicates with the backend through REST APIs, while the backend connects to MySQL for data storage.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The database was designed using relational database principles.

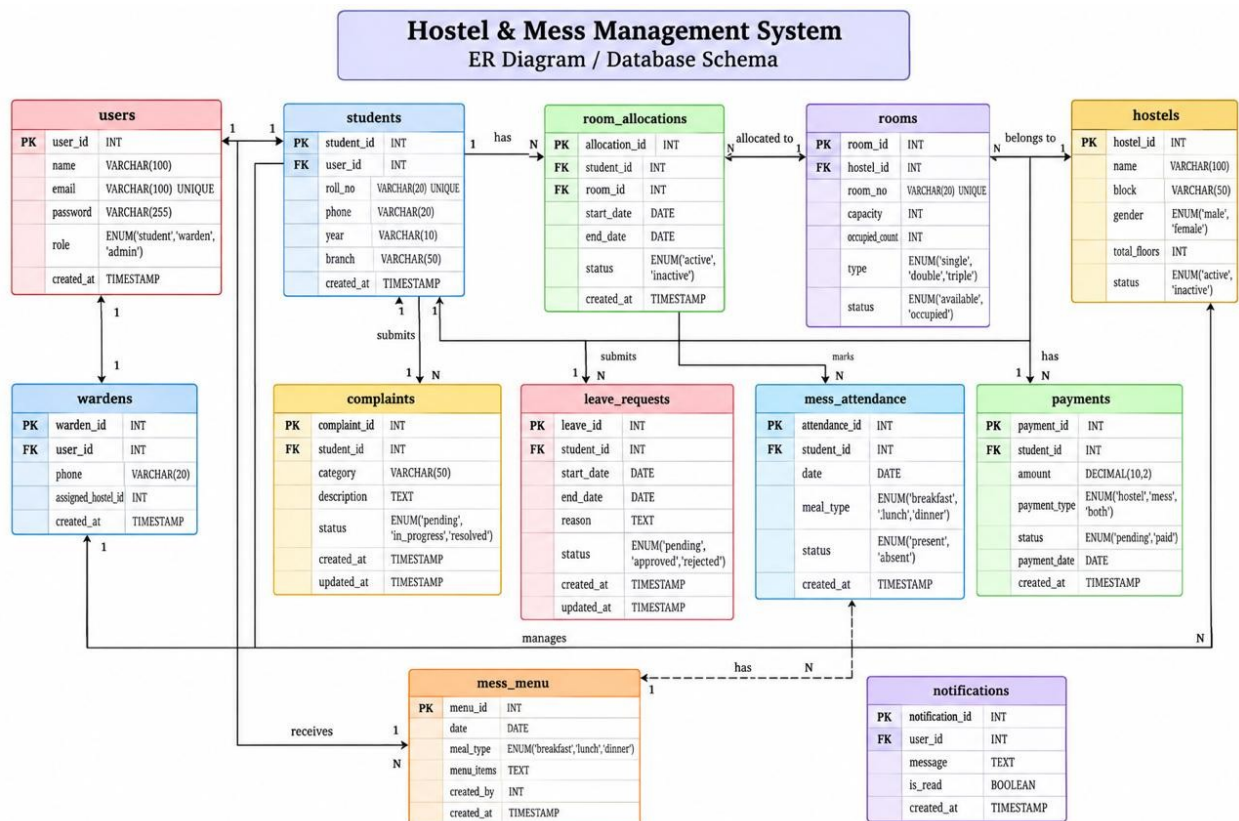
The system uses:

- Primary keys.
- Foreign keys.
- Unique constraints.
- ENUM fields for controlled statuses.
- NOT NULL constraints.
- Transactions for room allocation operations.
- Cascading deletion where appropriate.

For example, each student is associated with a user account, while room allocations connect students with rooms.

5.2 ER Diagram / Database Schema

The database consists of seven related tables, shown in Figure 5.1.



5.3 Design Decisions and Assumptions

The system separates authentication information from role-specific information.

The `users` table stores common login information and the user's role, while `students` and `wardens` store role-specific information.

Room allocation is maintained separately from room information so that previous allocations can be retained.

The current active room is identified using the allocation status.

Leave requests are connected to student records, allowing wardens and administrators to review requests along with student information.

Notifications are connected to users so that important events such as approved or rejected leave requests can be communicated to the appropriate student.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The Hostel and Mess Management System is divided into different modules for students, wardens, and administrators. The main modules include authentication, student management, hostel management, room allocation, mess management, complaints, leave requests, payments, notifications, and administration.

The authentication module manages user login and verifies the credentials of students, wardens, and administrators. After successful login, the system generates a secure JWT token that is used for accessing protected features.

The student module manages student profile information and allows students to view their hostel, room, mess, complaints, leave requests, payments, and notifications. The warden module provides functionality for managing students, rooms, complaints, and leave requests.

The administrator module provides centralized management of students, wardens, hostels, rooms, mess information, payments, and reports. All these modules communicate with the MySQL database through the Express.js backend.

6.2 Front-end Logic

The frontend is developed using React.js and Vite. It provides separate dashboards for students, wardens, and administrators. Each dashboard contains the features that are relevant to the user's role.

React Router is used to navigate between different pages such as the dashboard, profile, mess, complaints, leave requests, payments, and room information. Reusable components such as sidebars, navigation components, notifications, loading components, and protected routes are used throughout the application.

The frontend also uses an authentication context to maintain the login state of the user. Based on the user's role, the system displays the appropriate dashboard and prevents unauthorized access to restricted pages.

6.3 API Integration

The frontend communicates with the backend using REST APIs. Axios is used to send HTTP requests from the React application to the Node.js and Express.js backend.

When a user performs an operation, such as submitting a complaint or leave request, the frontend sends the required information to the corresponding API. The backend processes the request, communicates with the MySQL database, and sends the result back to the frontend.

The API service files are organized according to different modules such as authentication, students, mess, complaints, leaves, payments, notifications, wardens, and administrators. This makes the application easier to maintain and manage.

6.4 Authentication and Authorization

Authentication is implemented using JSON Web Tokens. When a user successfully logs in, the backend verifies the credentials and generates a JWT token. The token is then used to authenticate subsequent requests to protected APIs.

Authorization is implemented using role-based access control. Students, wardens, and administrators have different permissions within the system. For example, students can manage their own complaints and leave requests, while wardens can process complaints and leave requests and administrators can manage the overall hostel system.

Passwords are securely hashed before being stored in the database. Authentication and role-based middleware are used on the backend to ensure that only authorized users can access protected resources.

CHAPTER 7: FEATURES

7.1 List of Main Features

The Hostel and Mess Management System provides features for three main types of users: students, wardens, and administrators. Students can view their profile, room details, mess information, payments, notifications, complaints, and leave requests.

Wardens can manage students and rooms and can process complaints and leave requests. Administrators have access to the overall system and can manage hostels, rooms, students, wardens, mess information, payments, and reports.

7.2 How Each Feature Works

The system provides separate features based on user roles. Students can manage complaints, leave requests, mess attendance, payments, and room information. Wardens can manage rooms, allocations, complaints, and leave requests. Administrators can manage hostels, rooms, users, mess, payments, and reports. All operations are controlled through role-based authentication.

The screenshot displays the 'Payments' section of the Hostel Management system. The left sidebar contains navigation links: Dashboard, Students, Wardens, Hostels, Rooms, Mess, Payments (highlighted), and Reports. A 'Logout' button is at the bottom of the sidebar. The main content area is titled 'Payments' with the subtitle 'Manage student hostel and mess payments.' It features a 'Create Payment' form with the following fields: 'Student' (a dropdown menu showing 'Select Student'), 'Amount' (a text input with 'Example: 5000'), and 'Payment Type' (a dropdown menu showing 'Select Payment Type'). A blue 'Create Payment' button is at the bottom of the form. Below the form is a 'Payment List' table with the following data:

ID	Student	Roll Number	Amount	Type	Date	Status
1	Test Student	STU001	₹5000.00	hostel_fee	26/09/2026, 13:25:03	Paid

Hostel Management

Dashboard

Profile

Room

Mess

Complaints

Leave Requests

Payments

Logout

Complaints

Submit and track your hostel complaints.

Submit a Complaint

Category

Example: Room Maintenance

Description

Describe your complaint

Submit Complaint

Complaint History

ID	Category	Description	Status	Created At	Resolved At
2	Electrical	Light in the room is not working.	pending	2026-09-26T10:13:40.000Z	-
1	Room Maintenance	The room fan is not working properly.	resolved	2026-09-26T07:35:56.000Z	2026-09-26T07:41:03.000Z

CHAPTER 8: TESTING

8.1 Postman API Testing

The backend APIs were tested to verify that each module responds correctly to valid and invalid requests. Authentication, student, hostel, room allocation, mess, complaint, leave, payment, and notification APIs were tested during development.

The testing ensured that valid requests were processed correctly and invalid or unauthorized requests were rejected with appropriate responses.

8.2 Integration Testing

Integration testing was performed by connecting the React frontend with the Node.js and Express.js backend and the MySQL database. Different operations were tested from the frontend to verify that data was correctly sent to the backend and stored or retrieved from the database.

The complete flow of important operations such as login, room allocation, complaint submission, leave requests, payments, and notifications was verified.

8.3 Tools Used for Testing

The project was tested using API testing tools, the MySQL command-line client, and manual browser testing. API requests were used to verify backend functionality, while the MySQL client was used to check whether the expected data was correctly stored in the database.

Manual browser testing was also performed to verify page navigation, dashboards, authentication, role-based access, and the overall behavior of the frontend.

8.4 Test Cases and Results

A representative sample of the test cases executed, drawn from the project's actual test log, is summarized below.

No.	Test Case	Expected Result	Actual Result
1	Login with valid credentials	Login successful	Pass
2	Login with invalid password	Login rejected	Pass
3	Student accesses admin dashboard	Access denied/redirected	Pass
4	Warden accesses admin dashboard	Access denied/redirected	Pass
5	Admin accesses student dashboard	Access denied/redirected	Pass
6	Unauthenticated user accesses protected route	Redirect to login	Pass
7	Student submits complaint	Complaint created	Pass
8	Warden updates complaint	Status updated	Pass
9	Student submits leave request	Request created	Pass
10	Warden approves leave	Status updated and notification generated	Pass
11	Student views notification	Notification displayed	Pass
12	Student marks notification as read	Notification becomes read	Pass
13	Student marks mess attendance	Attendance recorded	Pass
14	Duplicate mess attendance	Duplicate rejected	Pass
15	Room allocation	Allocation created	Pass
16	Room transfer	Previous allocation completed and new allocation created	Pass
17	Payment status update	Payment status updated	Pass
18	Student views payment history	Payment displayed	Pass

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application can be started by first creating the MySQL database and importing the required schema and seed files. After configuring the database connection, the backend dependencies are installed using npm and the Express server is started.

The frontend dependencies are then installed and the React application is started using Vite. Once both servers are running, users can access the application through a web browser.

9.2 Environment Configuration

The backend uses an environment configuration file to store database and authentication settings. Database host, username, password, database name, server port, and JWT secret are maintained through environment variables.

The `.env` file is excluded from Git using `.gitignore` so that sensitive database credentials and authentication secrets are not uploaded to GitHub.

9.3 Hosting of Frontend and Backend

The current version of the project is configured for local development and demonstration. The React frontend runs using the Vite development server, while the Node.js and Express.js backend runs separately.

The backend communicates with the local MySQL database through the configured database connection. The application can later be deployed to cloud hosting platforms for public access.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

One of the major challenges during development was maintaining correct relationships between the different database tables. The system contains several related entities such as students, rooms, hostels, complaints, leave requests, payments, and notifications.

Another challenge was implementing authentication and role-based authorization. Different users require different levels of access, so protected routes and middleware had to be implemented carefully.

10.2 Solutions Applied

Database relationships were maintained using primary keys and foreign keys. For example, leave requests are connected to students using the `student_id` foreign key. This ensures that each leave request belongs to the correct student.

JWT authentication and role-based middleware were implemented to protect backend APIs. The frontend also uses protected routes to prevent unauthorized users from accessing restricted pages.

10.3 Current Limitations or Known Bugs

The current version is mainly intended for academic demonstration and local use. It does not currently include advanced features such as mobile applications, biometric mess attendance, online payment gateway integration, or automated SMS and email notifications.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

The system can be enhanced in the future by adding online payment integration, QR-based mess attendance, biometric attendance, automated email notifications, password recovery, visitor management, and hostel maintenance management.

A mobile application can also be developed so that students, wardens, and administrators can access the system from smartphones.

11.2 Possible Integrations or Optimizations

The application can be deployed on cloud infrastructure to make it accessible from different locations. Real-time notifications can be implemented using WebSockets, while caching and optimized database queries can be introduced to improve performance when the number of users increases.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

The Hostel and Mess Management System provides a centralized platform for managing hostel and mess-related activities. It replaces several manual processes with a web-based system that connects students, wardens, and administrators.

The application integrates React.js, Node.js, Express.js, and MySQL to provide a complete full-stack solution for hostel management.

12.2 What Was Achieved

The project successfully implements student management, hostel and room management, room allocation, mess management, complaint management, leave requests, payments, notifications, authentication, and role-based authorization.

The system provides separate interfaces for students, wardens, and administrators, allowing each user to access the features required for their responsibilities.

12.3 Skills Learned

Through this project, practical knowledge was gained in React.js, Node.js, Express.js, MySQL, REST API development, database relationships, JWT authentication, role-based authorization, Axios, Git, GitHub, and full-stack web application development.

The project also provided practical experience in connecting a frontend application with a backend server and relational database.

CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev>
- Vite — Official Documentation, <https://vitejs.dev>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- Express.js — Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- mysql2 (Node.js MySQL driver) — npm documentation, <https://www.npmjs.com/package/mysql2>
- jsonwebtoken — npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- bcrypt — npm documentation, <https://www.npmjs.com/package/bcrypt>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs — JavaScript and Web APIs reference, <https://developer.mozilla.org>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

```
</> JavaScript

const token = jwt.sign(
  {
    userId: user.id,
    role: user.role
  },
  process.env.JWT_SECRET,
  {
    expiresIn: "1d"
  }
);
```

Appendix B: Installation and Setup Instructions

- `mysql -u root -p < database/schema.sql`
- `mysql -u root -p < database/seed.sql`
- `cd backend && npm install && node server.js` (backend, port 5002)
- `cd frontend && npm install && npm run dev` (frontend, port 5173)
- Open `http://localhost:5173` in a browser

Appendix C: Default Login Credentials

Role	Email	Password
Admin	admin@hostel.com	admin123
Warden	warden@hostel.com	warden123
Student	student@test.com	123456

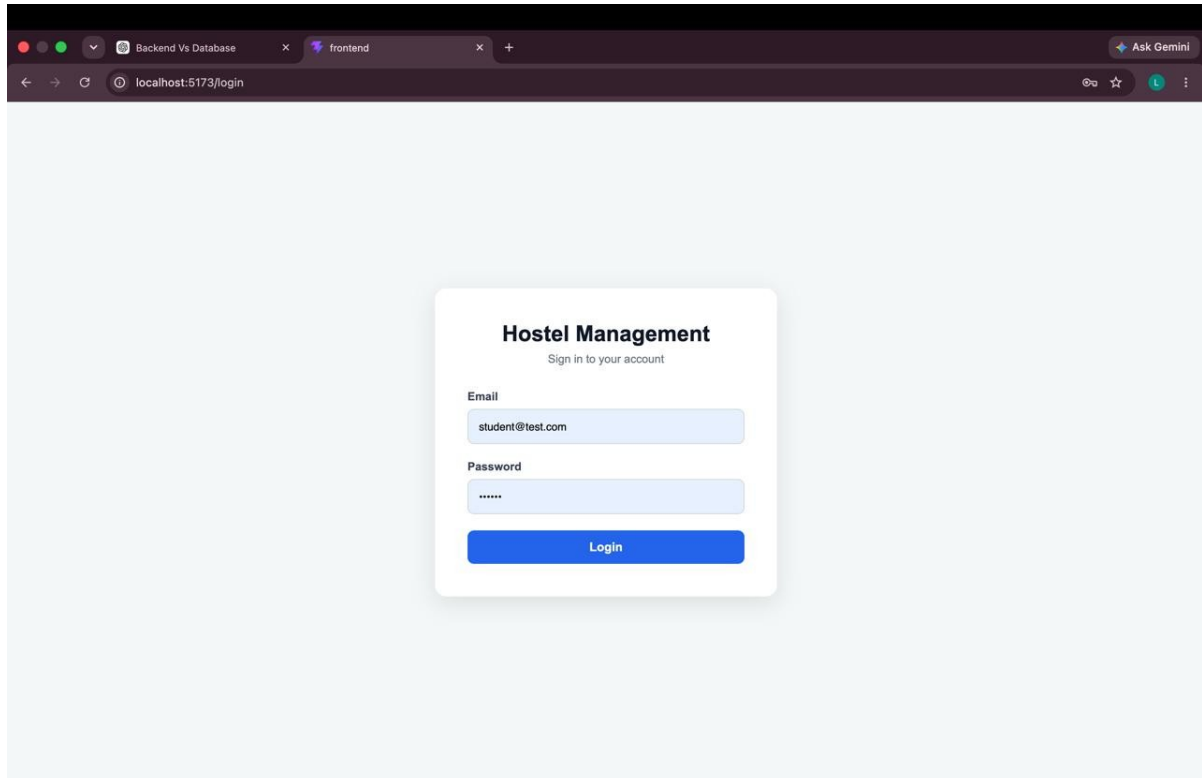
Appendix D: API Reference Summary

Module	Method	Endpoint	Role
Auth	POST	/api/auth/register	Public
Auth	POST	/api/auth/login	Public
Student	GET	/api/students/profile	Student
Hostel	GET	/api/hostels	All authenticated roles
Hostel	GET	/api/hostels/rooms	All authenticated roles
Hostel	POST	/api/hostels	Admin
Hostel	POST	/api/hostels/rooms	Admin/Warden
Allocation	POST	/api/room-allocations	Admin/Warden
Allocation	GET	/api/room-allocations	Admin/Warden
Allocation	PATCH	/api/room-allocations/:id/deallocate	Admin/Warden
Allocation	PATCH	/api/room-allocations/:id/transfer	Admin/Warden
Mess	GET	/api/mess/menu	All authenticated roles
Mess	POST	/api/mess/menu	Admin/Warden
Mess	POST	/api/mess/attendance	Student
Mess	GET	/api/mess/attendance/my	Student
Complaints	POST	/api/complaints	Student
Complaints	GET	/api/complaints/my	Student
Complaints	GET	/api/complaints	Warden/Admin
Complaints	PATCH	/api/complaints/:id/status	Warden/Admin
Leaves	POST	/api/leaves	Student
Leaves	GET	/api/leaves/my	Student
Leaves	GET	/api/leaves	Warden/Admin
Leaves	PATCH	/api/leaves/:id/status	Warden/Admin
Payments	GET	/api/payments/my	Student
Payments	POST	/api/payments	Admin
Payments	PATCH	/api/payments/:id/status	Admin
Notifications	GET	/api/notifications	Authenticated
Notifications	PATCH	/api/notifications/:id/read	Authenticated
Notifications	PATCH	/api/notifications/read-all	Authenticated

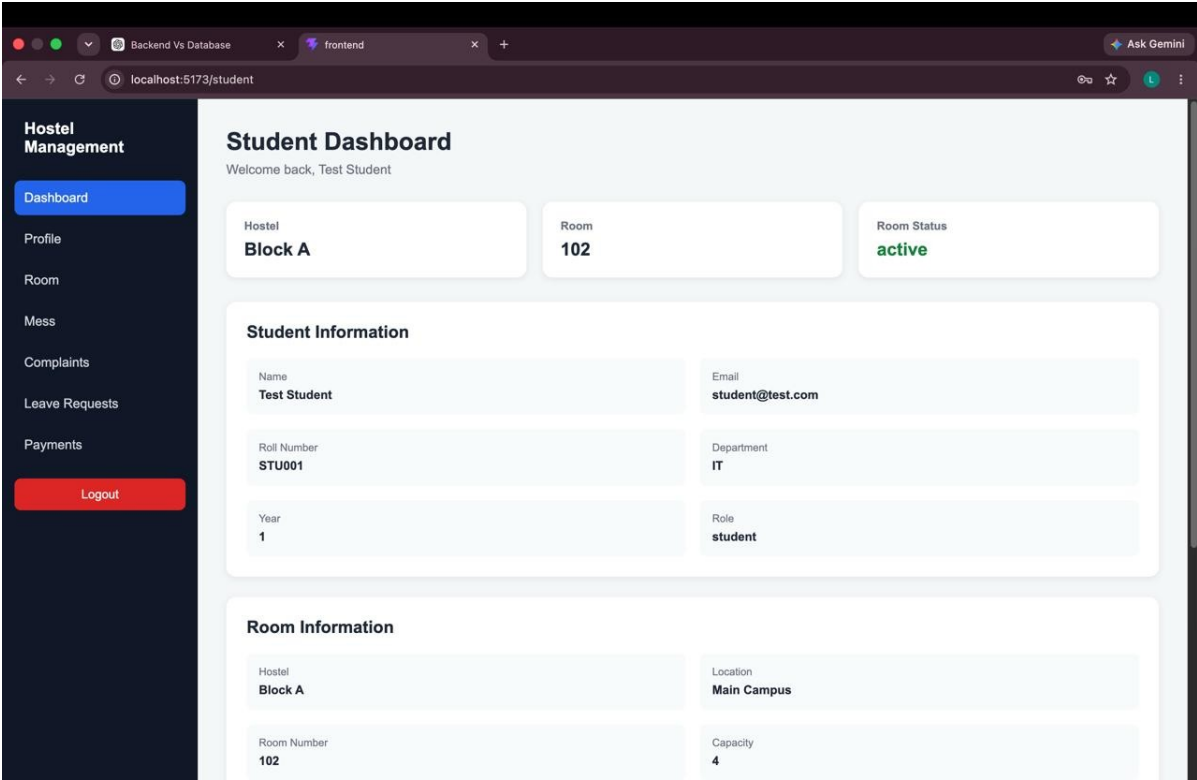
Appendix E: Screenshots

The following pages are left for the student to complete with actual screenshots taken from the running application before submission. Suggested screenshots, in order:

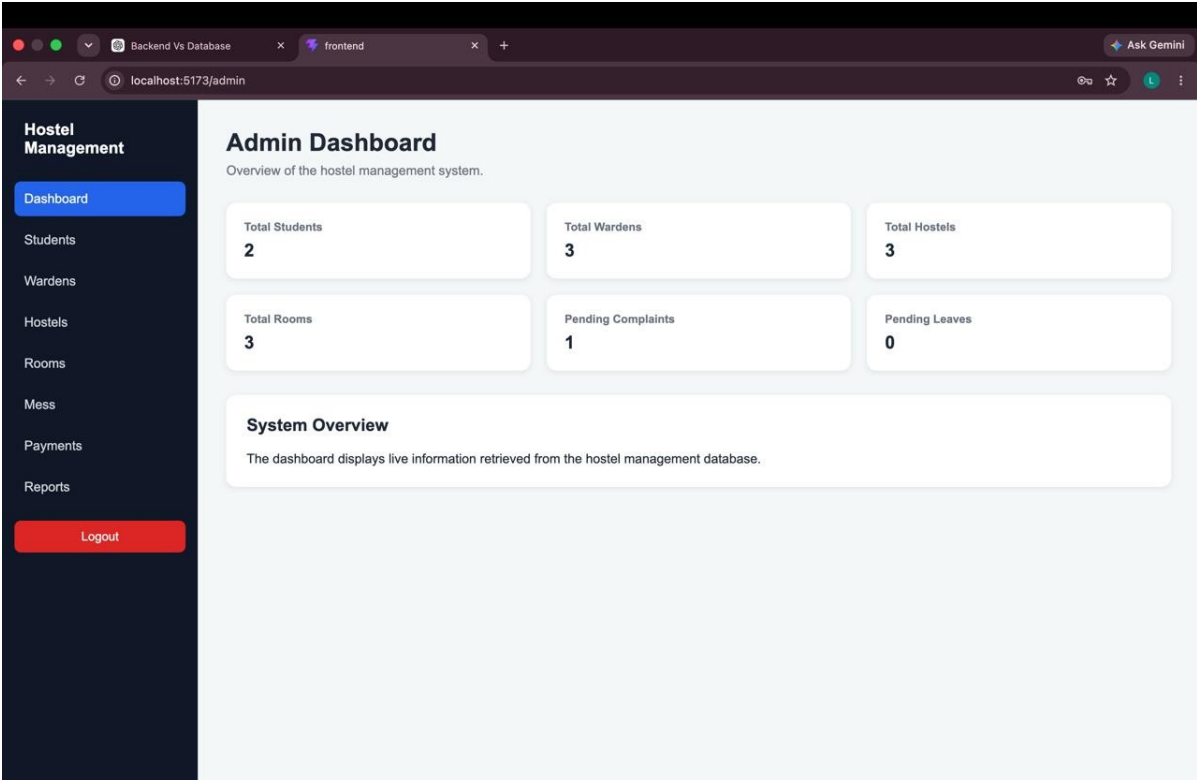
1. Login Page



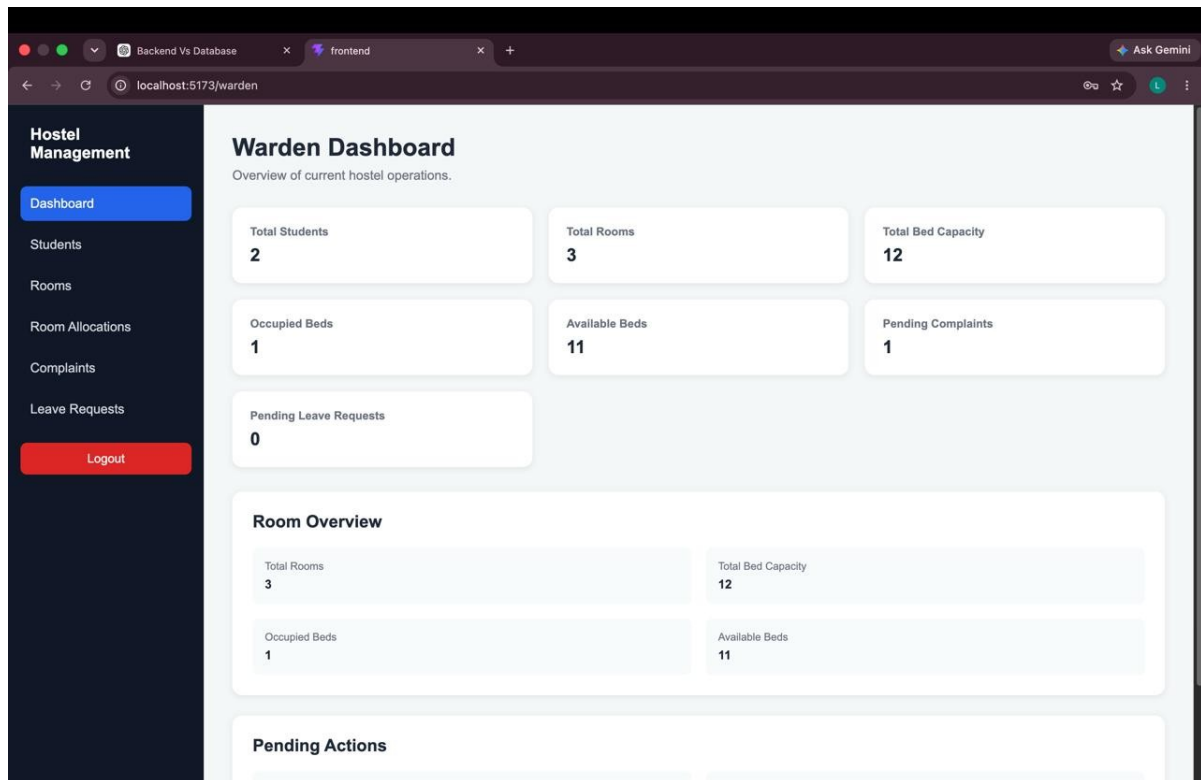
2. Student Dashboard



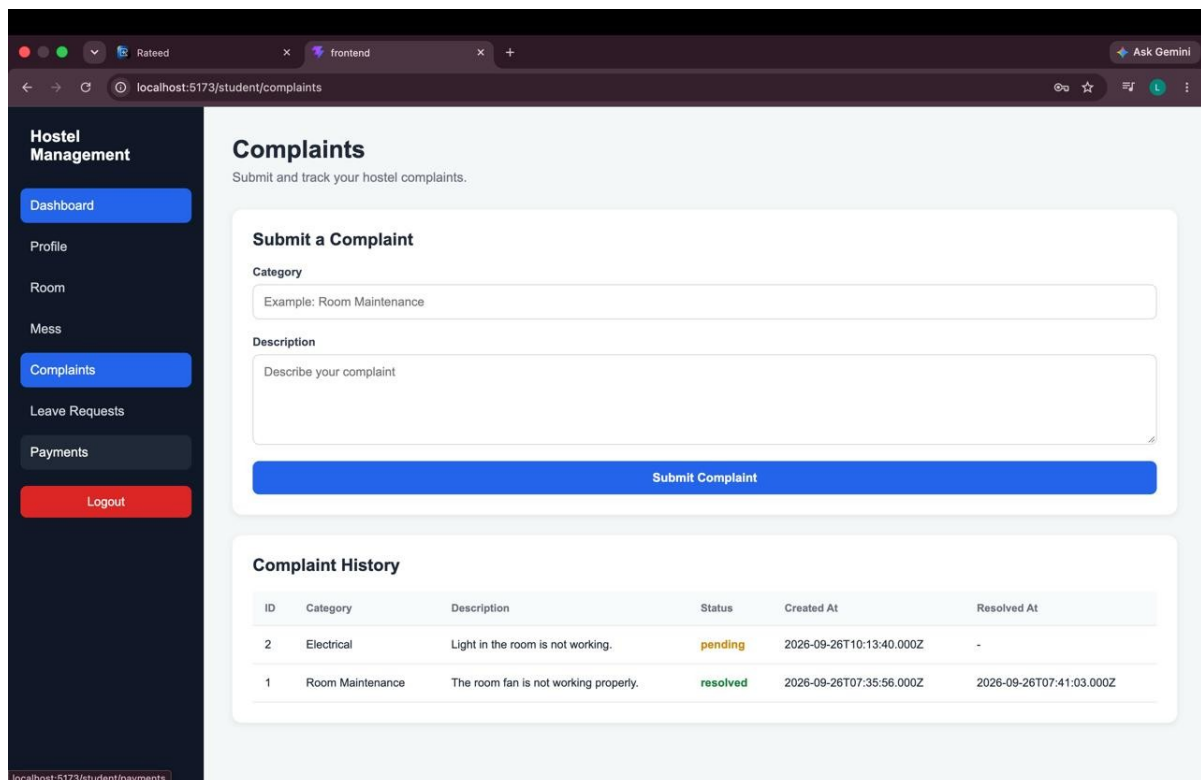
3. Admin Dashboard



4. Warden Dashboard



5. Complaints Page



6. Payments Page

The screenshot shows the 'Payments' page of the Hostel Management system. The left sidebar contains navigation links: Dashboard, Students, Wardens, Hostels, Rooms, Mess, Payments (highlighted), and Reports. A 'Logout' button is at the bottom. The main content area is titled 'Payments' with the subtitle 'Manage student hostel and mess payments.' It features a 'Create Payment' form with fields for Student (a dropdown), Amount (with an example of 5000), and Payment Type (a dropdown). A blue 'Create Payment' button is at the bottom of the form. Below the form is a 'Payment List' table with columns: ID, Student, Roll Number, Amount, Type, Date, and Status. The table contains one entry: ID 1, Student Test Student, Roll Number STU001, Amount ₹5000.00, Type hostel_fee, Date 26/09/2026, 13:25:03, and Status Paid (with a dropdown arrow).

ID	Student	Roll Number	Amount	Type	Date	Status
1	Test Student	STU001	₹5000.00	hostel_fee	26/09/2026, 13:25:03	Paid

7. Leave Requests

The screenshot shows the 'Leave Requests' page of the Hostel Management system. The left sidebar contains navigation links: Dashboard, Students, Rooms, Room Allocations, Complaints, Leave Requests (highlighted), and Logout. The main content area is titled 'Leave Requests' with the subtitle 'Review and manage student leave requests.' It displays four summary cards: Total Requests (2), Pending (0), Approved (2), and Rejected (0). Below these is a 'Leave Request List' table with columns: ID, Student, Roll Number, Start Date, End Date, Reason, Status, and Action. The table contains two entries: ID 2, Student Test Student, Roll Number STU001, Start Date 2026-10-10, End Date 2026-10-12, Reason Personal work, Status approved, and Action Approved (with a dropdown arrow); and ID 1, Student Test Student, Roll Number STU001, Start Date 2026-10-01, End Date 2026-10-03, Reason Going home for a family function, Status approved, and Action Approved (with a dropdown arrow).

ID	Student	Roll Number	Start Date	End Date	Reason	Status	Action
2	Test Student	STU001	2026-10-10	2026-10-12	Personal work	approved	Approved
1	Test Student	STU001	2026-10-01	2026-10-03	Going home for a family function	approved	Approved

8. Students Page

The screenshot shows a web application interface for a Hostel Management system. The browser's address bar displays `localhost:5173/warden/students`. The left sidebar, titled "Hostel Management", contains a list of navigation links: "Dashboard", "Students" (highlighted in blue), "Rooms", "Room Allocations", "Complaints", and "Leave Requests". At the bottom of the sidebar is a red "Logout" button. The main content area is titled "Students" and includes a subtitle "View all students and their current room allocations." Below this, a white box displays "Total Students" as **2**. A "Student List" table follows, containing two rows of student data.

ID	Name	Roll Number	Email	Department	Year	Hostel	Room
1	Test Student	STU001	student@test.com	IT	1	Block A	102
2	Second Student	STU002	student2@test.com	CSE	1	Not allocated	Not allocated